

2. Communication

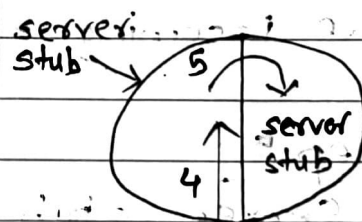
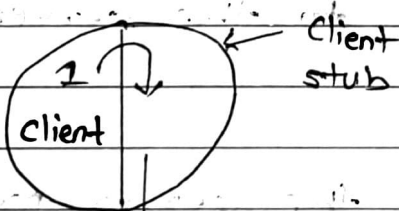
Page No.

Date:

Q.1 What is Remote Procedure Call? - Describe the working of RPC in detail.

- ⇒
- Program executing on machine A calls procedure or functions that is located at machine B is called RPC.
 - The primitives used in sending and receiving the messages do not hide communication.
 - In RPC, calling program calls remote procedure by sending parameters in call and callee returns the result of the procedure.
 - RPC enables inter-process communication across different systems in distributed environment.
 - Working :
 - The following figure shows working of the RPC -

Client CPU Server CPU



Operating System

Operating system

3

Network

Step 1:

⇒ client calls the client stub. As client stub is on client machine, this call is a local Procedure Call (LPC) and the parameters pushed onto the stack in the normal manner.

Step 2:

⇒ Client stub packs the parameters into a message and issues a system call to send the message.

- This process is called marshalling.

Step 3:

⇒ Kernel sends the message from the client machine to the server machine.

Step 4:

⇒ The kernel on the server machine, send the incoming packet to the server stub.

Step 5:

⇒ Server stub calls the server procedure.

- The server then handover the result to server stub which packs it into message.
- Kernel then sends the message to client where kernel at client handover it to the client stub.
- Client stub unpack the message and handover result to client.

• Parameter passing in RPC

1) Call by value

⇒ All parameter's are copied into a message that is transmitted from the client to the server although through the network.

- It is time consuming for passing large datatypes such as trees and arrays.

2) Call by reference

⇒ A pointer is meaningful only within the address space of the process in which it is being used.

- If an address 2000 on the client is passed to the server as parameter, then address 2000 on point to some other value on the server machine.

3) Call by copy - restore

⇒ Copy the array into the message and send it to the server.

- The server stub can then call the server with a pointer to this array, even though this pointer has a different numerical value than the sender.

• Transparency !

- To achieve the goal of transparency, implementation of RPC is based on the concept of STUBS.
- It provides a normal procedure call abstraction by concealing from programs the interface to the underlying RPC system.
- RPC involves two processes: Client & Server.
- Hence to achieve transparency, a separate stub process is associated with each of these two processes.
- Also, to hide the existence and functional details of the underlying network, RPC communication package called: RPC Runtime is used on the both sides.

Q.2 Explain the steps involved in the RMI execution process in detail.

- ⇒
- Remote Invocation Method (RMI) is an API that allows an object to invoke a method on an object that exists in another address space, which could be on the same machine or on a remote machine.
 - Through RMI, an object running in a JVM present on computer can invoke methods on an object present in another JVM.
 - RMI creates a public remote server object that enables client and server-side communication through simple method calls on the server object.
 - Stub Object: The stub object on the machine builds an information block and sends this information to the server.
 - Skeleton Object: The skeleton object passes the request from the stub object to the remote object.
 - Steps involved in RMI are:
 - 1) Defining a remote interface
 - 2) Implementing the remote interface
 - 3) Creating stub and skeleton objects from the implementation class using `rmic`.
 - 4) Start the `rmiregistry`
 - 5) Create and execute server app program
 - 6) Create and execute client app program

1) Defining the remote interface

⇒ For creating the remote interface, extend the Remote Interface and declare the RemoteException with all the methods of the remote interface.

— Here, we are creating a remote interface that extends the Remote interface.

— There is only one method named add() and it declares RemoteException.

2) Implementing the remote interface

⇒ The next step is to implement the remote interface.

— To implement the remote interface, the class should extend to UnicastRemoteObject class of java.rmi package.

— Also, a default constructor needs to be created to throw the java.rmi.RemoteException from its parent constructor in class.

3) Creating stub and skeleton object

⇒ Next step is to create stub and skeleton objects using the rmi compiler.

— The rmic tool invokes the RMI compiler and creates stub and skeleton objects.

rmic AdderRemote

rmic SearchQuery

4) Start the rmiregistry

⇒ Now, start the registry service by using the rmiregistry tool.

- If we don't specify the port number, it uses a default port number.

- For e.g.,

```
rmiregistry 5000
```

5) Create and execute server application

⇒ The next step is to create the server application program and execute it on a separate command prompt.

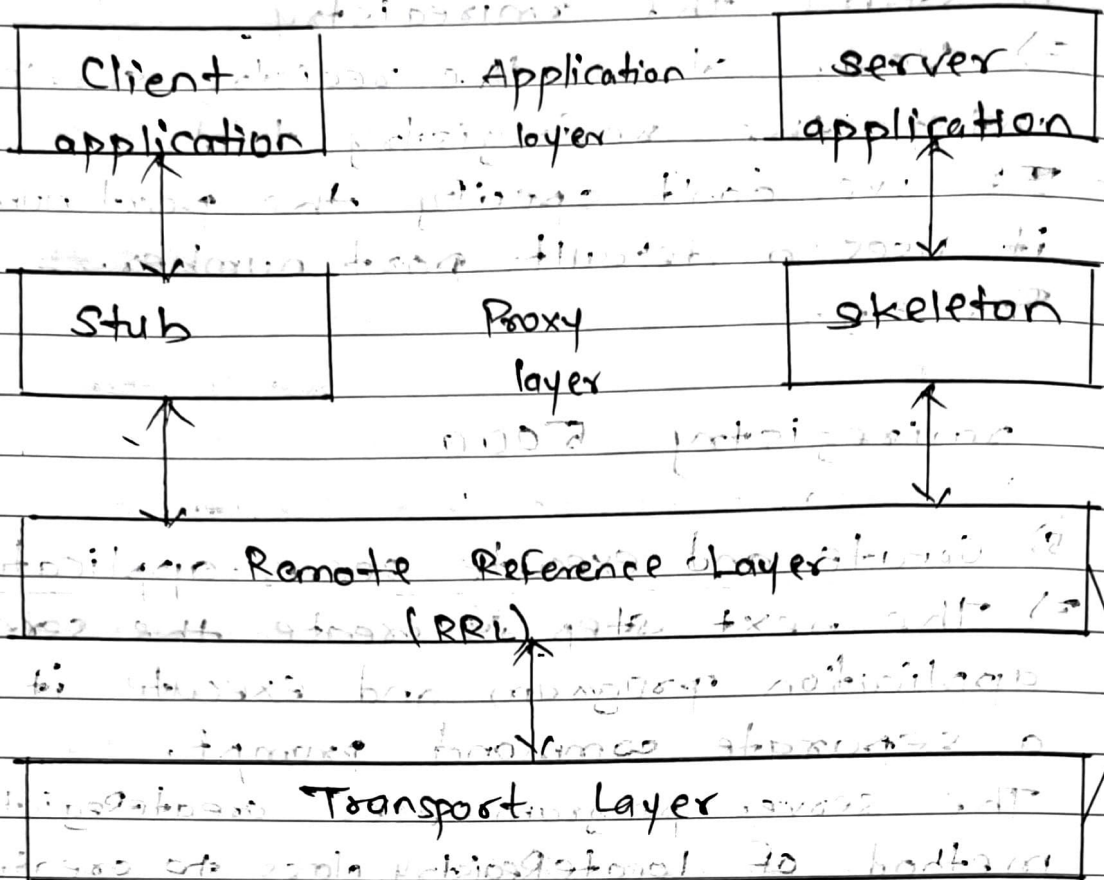
- The server program uses createRegistry method of LocateRegistry class to create rmiregistry within ~~this~~ the server JVM with the port number passed as an argument.

- The rebind method of Naming class is used to bind the remote object to the new name.

6) Create and execute client application

⇒ The last step is to create the client application program and execute it on a separate command prompt.

- The lookup method of the Naming class is used to get the reference of the stub object.



Q.3 Difference between RMI and RPC.

RPC	RMI
1) RPC is a library and OS dependent platform.	1) RMI is a java platform.
2) RPC supports procedural programming.	2) RMI supports object-oriented programming.
3) RPC is less efficient in comparison with of RMI.	3) While RMI is more efficient than RPC.
4) RPC creates more overhead.	4) While it creates less overhead than RPC.
5) The parameters which are passed in RPC are ordinary or normal data.	5) While in RMI, objects are passed as parameters.
6) RPC does not provide security.	6) While RMI provide client-level security.
7) It's development cost is huge.	7) While it's development cost is fair or reasonable.

RPCRMI

8) There is multiple codes are needed for simple application in RPC.

8) While there is multiple codes are not needed for simple application in RMI.

9) Can be complex due to low-level implementation.

9) Generally simpler to use and implement.

Q.4 Explain various forms of message-oriented communication with suitable example.

⇒

- Message-oriented communication is a fundamental technique used in distributed systems where components exchange data in the form of message rather than direct method calls.
- It provides loose coupling and better fault tolerance across system.
- Types of communications:-

1) Persistent Communications

⇒ In this, the message is stored by communication system until it can be delivered to the receiver, even if the receiver is offline at the time of sending.

2) Transient Communication

⇒ The message is only stored as long as the sender and receiver are executing. If problem occurs, the message is simply discarded.

3) Asynchronous Communication

⇒ A sender continues with other work immediately upon sending a message to the receiver.

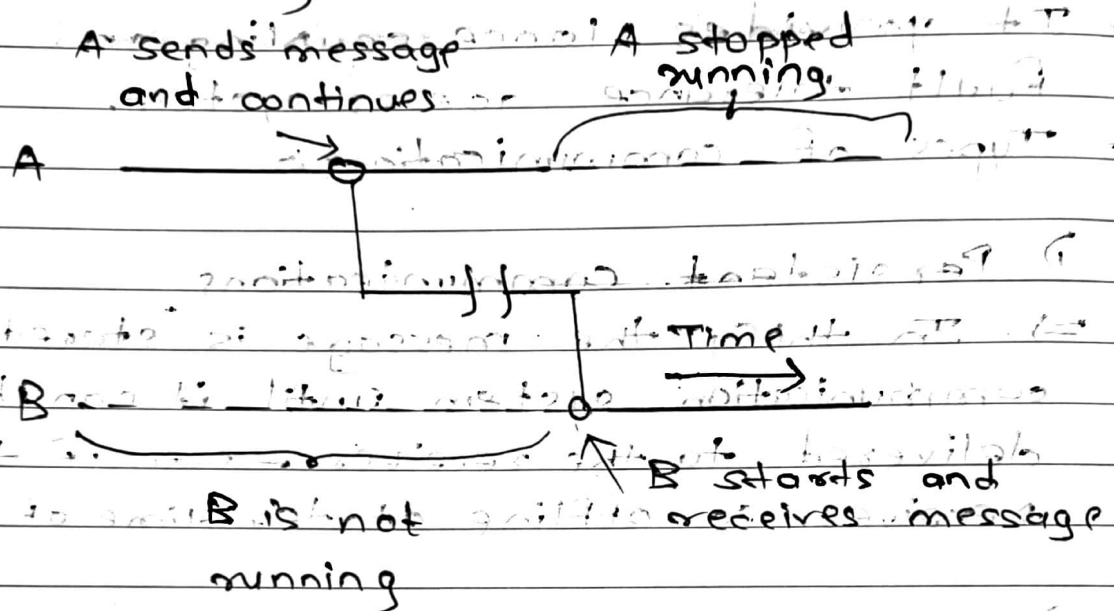
4) Synchronous communication

⇒ A sender blocks, waiting for reply from the receiver before doing any other work.

Following are the combinations of different types of communications :-

1) Persistent asynchronous communication

⇒ It refers to the communication model where messages are sent and persistently stored by the system until they are successfully delivered to the recipient.

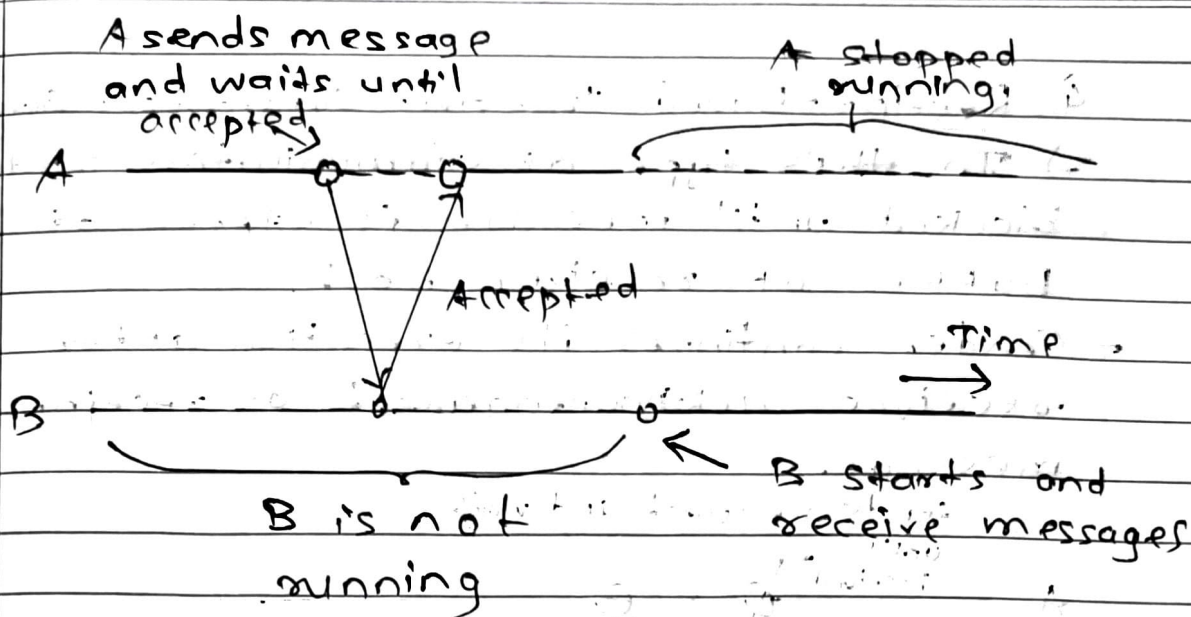


The sender does not wait for an immediate response and can continue execution after sending the message.
For e.g., Email.

2) Persistent synchronous communication

⇒ It refers to the communication where the sender and receiver remain blocked until the message is delivered and acknowledged and the message remains stored by the system until it is successfully received.

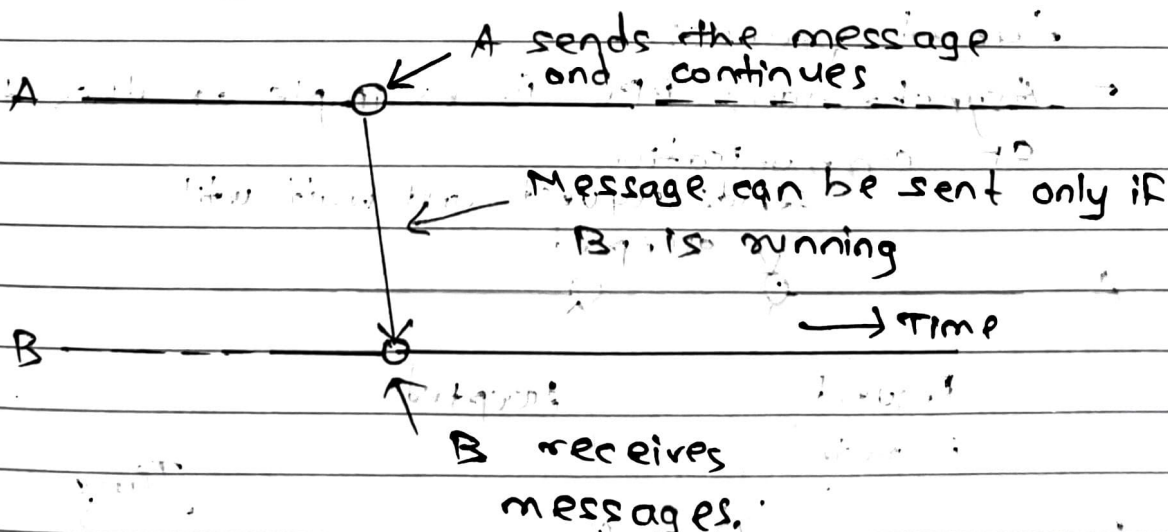
For e.g., RPC in client-server system



3) Transient asynchronous communication

⇒ It refers to the communication model where messages are sent and delivered without being persistently stored.

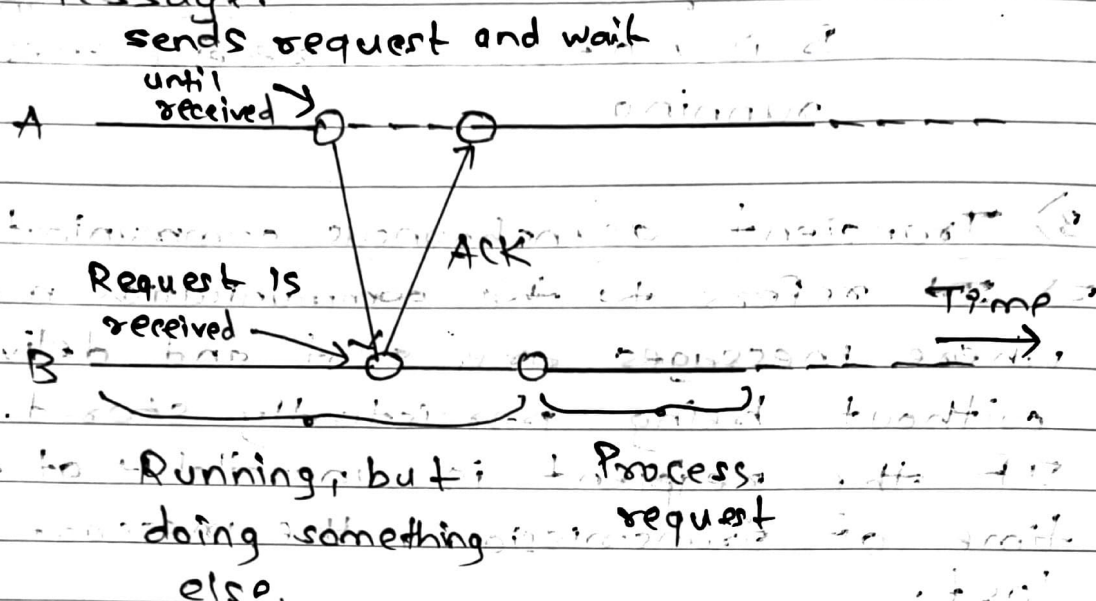
- If the recipient is unavailable at the time of transmission, the message is lost.
- The sender does not wait for an acknowledgement and continues execution after sending the message.
- For e.g. UDP



4) Receipt-Based Transient Synchronous Communication

⇒ In this type of communication, sender is blocked until message is stored at local buffer of receiver host.

- Sender continues its execution after it receives acknowledgement of receipt of message.

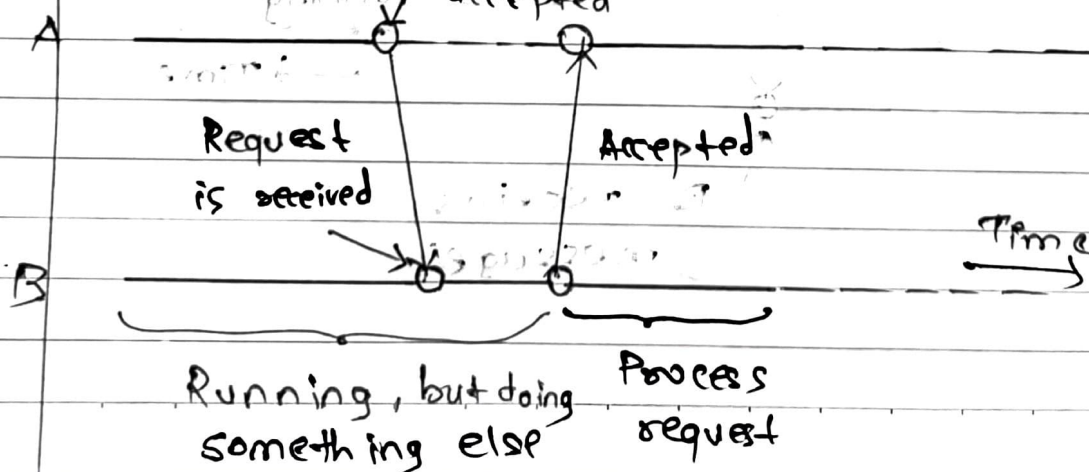


5) Delivery-based transient synchronous communication

⇒ In this type of communication, sender is blocked until message is delivered to the target process.

- Asynchronous RPC is example of this type of communication.

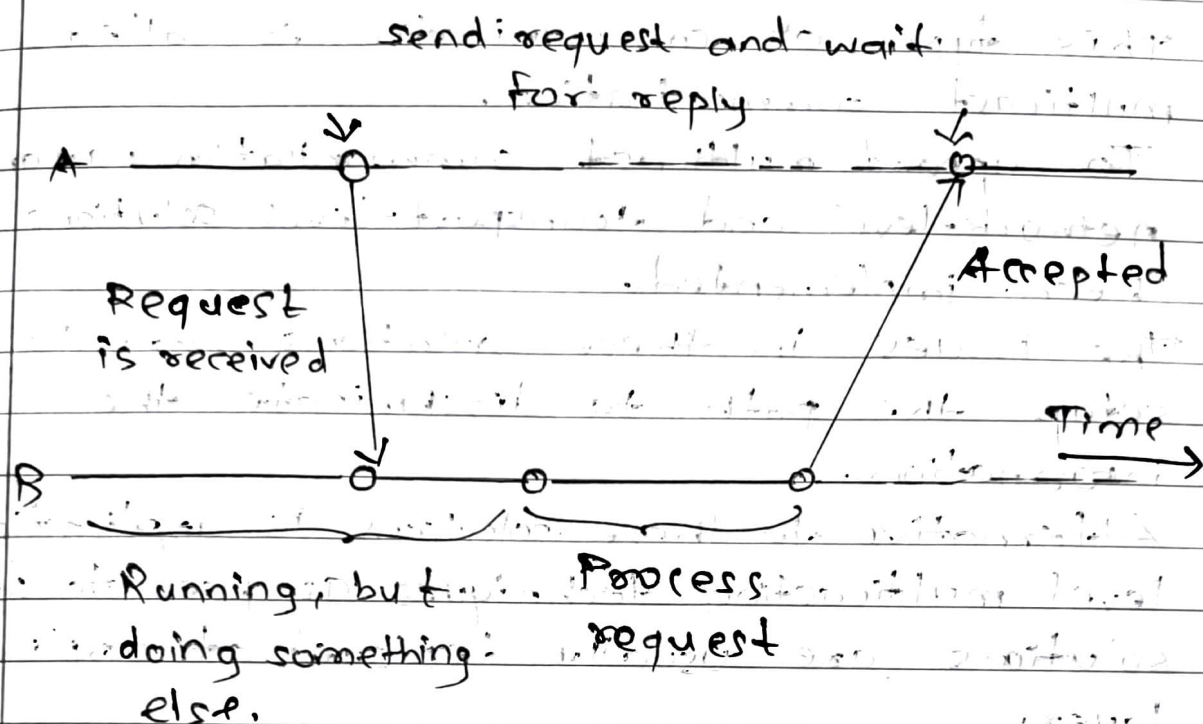
sends request and wait until



6) Response-biased transient synchronous communication

⇒ In this, sender is blocked until response is received from receiver process.

→ RPC and RMI sticks to this type of communication.



Q.5 - Explain Group Communication:

⇒

- In distributed system, it is necessary to have support for sending data to multiple receivers.
- This type of communication is called multicast communication.
- To support multicast communication, many network-level and transport-level solutions have been implemented.
- The issues in these solutions were to setup the path to discriminate the information.
- Alternative to these solutions is application level multicasting techniques as peer-to-peer solutions are usually deployed at application layer.
- It now became easier to setup communication paths.
- Characteristics:

1) Fault tolerance on replicated service

⇒ Replicated service is available on group of servers.

- Client requests are multicast to all these servers in group.

2) Better performance

⇒ Replication improves performance.

- If in some cases, replica of data is placed in client machine then changes in data item is multicast to all the processes managing the replicas.

3) Event notification

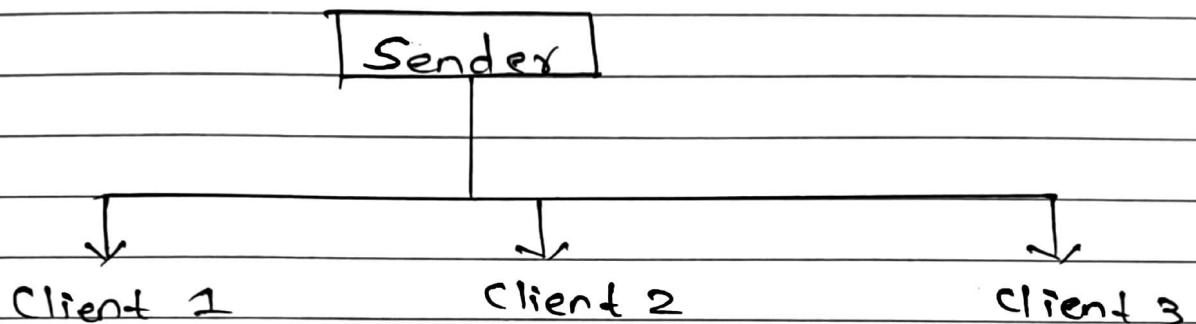
⇒ Multicast to group to notify if some event occurs.

- Types :-

1) 1:M Communication

⇒ A single sender (1) communicates with multiple receivers (M) simultaneously.

- Also called as multicast or broadcast.



- It is particularly useful ~~when~~ in distributed systems where broadcasting information to a group of nodes is necessary.

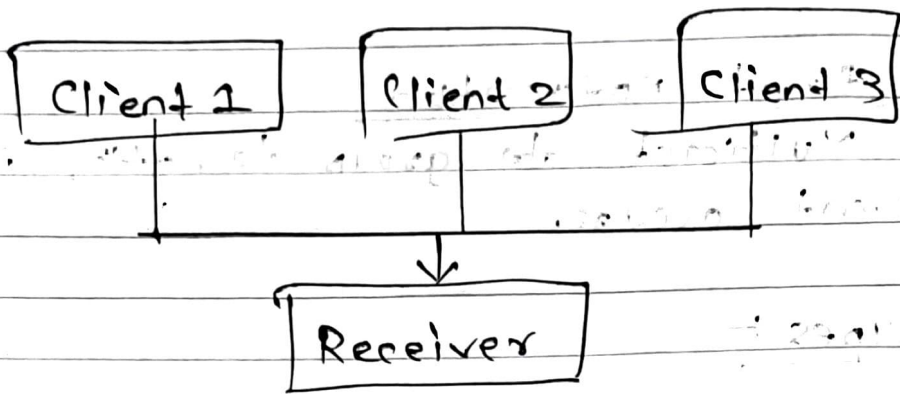
- For e.g.,

A chat server sending a message to all group members.

2) M:1 Communication

⇒ Multiple senders (M) send message to a single receiver (1). Also known as convergecast.

- Useful in data aggregation, logging, voting or monitoring.



For e.g.,

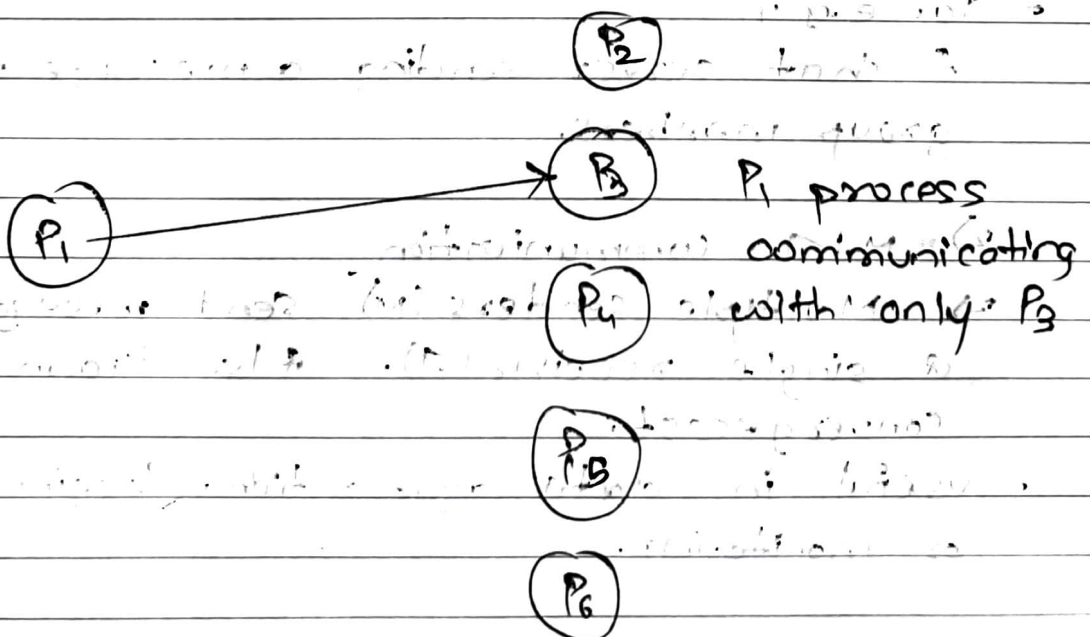
IoT sensors sending data to central server.

3) 1:1 (Unicast) Communication

⇒ Unicast communication is the point-to-point transmission of data between two nodes in a network.

Unicast is when a sender sends a message to specific recipient using their unique network address.

Each message targets one recipient, creating a direction connection between the sender and receiver.



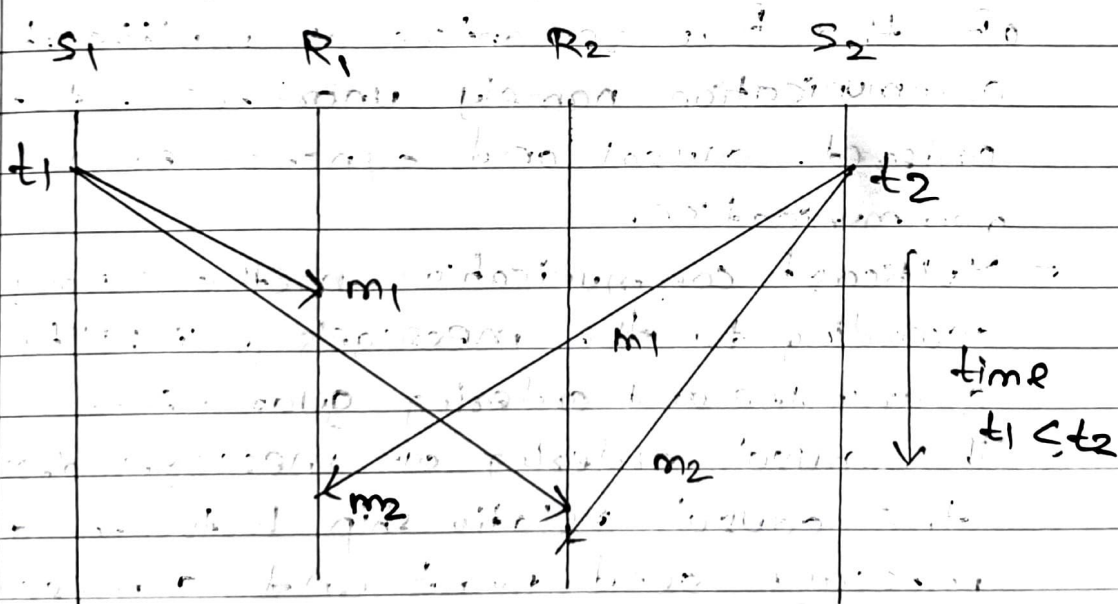
Q.6 Explain Absolute ordering and causal ordering process with help of an example for many-to-many communications.

=>

1) Absolute Ordering

=> All messages in the system are delivered in the same global order to all processes, regardless of when or where they were sent.

- It enforces a single total sequence of messages that every process agrees on.



- It ensures that all messages delivered to all receiver in exact in which they are sent.

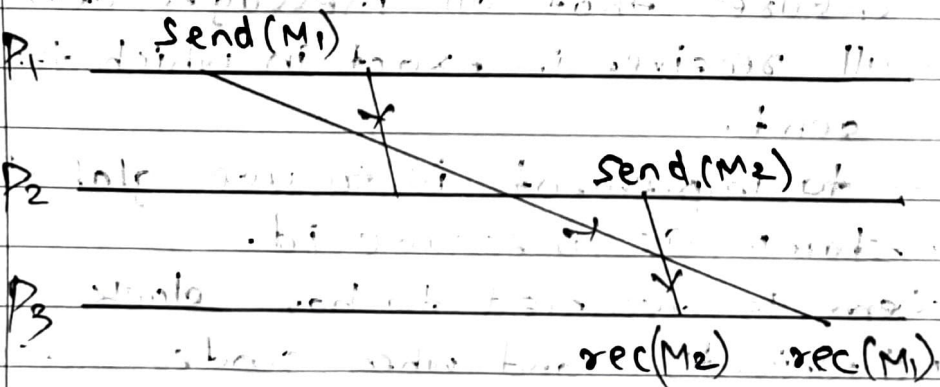
- Used to implement is to use global timestamp as message id.

- System is assumed to have clock synchronization and when sender message timestamp is taken as id of message and embedded in message.

- Kernel of receiver m/c saves all incoming messages in separate queue.
- A sliding window is used to periodically deliver message from receiver i.e. fixed time interval is selected as window size.
- Window size is properly chosen by considering maximum time for messages to kernel from one m/c to other in n/w.

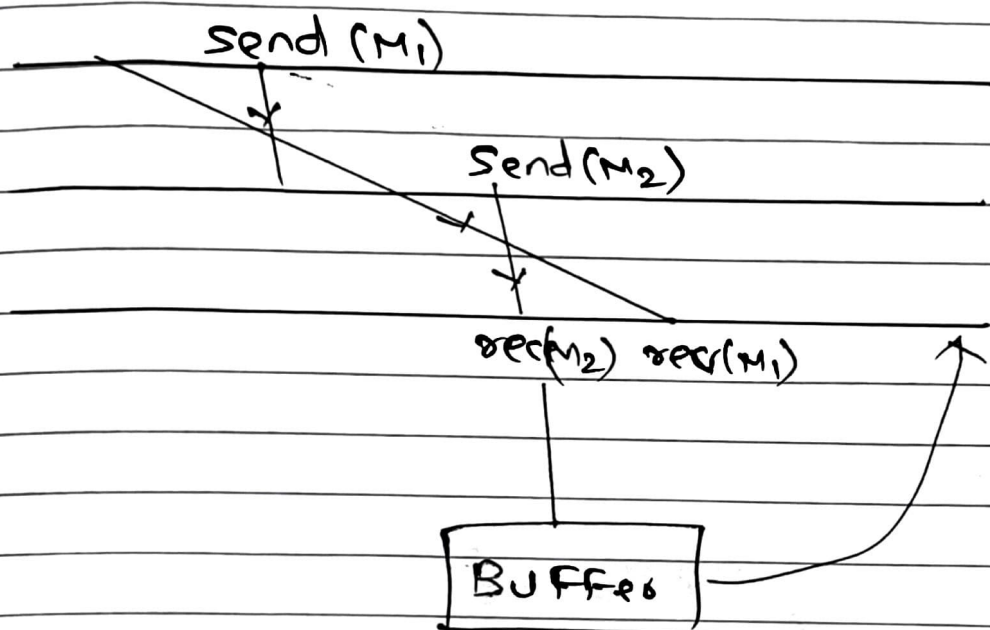
2) Causal Ordering

- Causal ordering of messages is one of the four semantics of multicast communication namely unordered, totally ordered, causal and sync-ordered communication.
- Multicast communication methods vary according to the message's reliability guarantee and ordering guarantee.
- The causal ordering of messages describes the causal relationship between a message send event and a message receive event.



- For example, if $\text{send}(M_1) \rightarrow \text{send}(M_2)$ then every recipient of both the messages M_1 and M_2 must receive the message M_1 before receiving the message M_2 .

In distributed systems the causal ordering of message is not automatically guaranteed.



DC MOD - 2

Q.7 Explain stream-oriented communication.
⇒

- Stream-oriented communication refers to the continuous flow of data between two or more processes over a network.

- Characteristics:

i) Streams are unidirectional.

ii) There is generally a single source.

• Support For Continuous Media

- Support for the exchange of time-dependent information is often formulated as support for continuous media.

- A medium refers to the means by which information is conveyed.

- These includes storage and transmission media, presentation media such as a monitor and so on.

• Data Stream :

- To capture the exchange of time-dependent information, distributed systems generally provide support for data streams.

- A data stream is nothing but a sequence of data units.

- Data streams can be applied to discrete as well as continuous media.

- For eg: UNIX pipes or TCP/IP connections are example of discrete data streams.

• Streams and Quality of Service (QoS) :

- Timing requirements are generally expressed as QoS requirements.
- These requirements describe what is needed from the underlying distributed system and network to ensure that
- QoS for continuous data streams mainly concerns timeliness, volume and reliability.
- Characteristics :

- i) The required bit rate at which data should be transported.
- ii) The maximum delay until a session has been setup.
- iii) The maximum end-to-end delay.
- iv) The maximum delay variance.
- v) The maximum round-trip delay.

Q.8 Difference between message-oriented and stream-oriented communication.

=>

Message-oriented	Stream-oriented
1) Data is sent as a discrete message.	1) Data is sent as continuous message.
2) Message is sent in chunks or packets.	2) Data Flows continuously without predefined boundaries.
3) Communication is connection-less, data is sent without any setup.	3) Communication is oriented, connection is established before comm.
4) It is unreliable best effort delivery without acknowledgement.	4) It is reliable, data acknowledged.
5) Re-transmission is not performed.	5) Lost data is reframe automatically.
6) Low overhead.	6) High overhead.
7) Transmission speed is very high.	7) Transmission speed is lower.
8) Suitable for applications that require discrete data exchange, such as chat apps or notifications.	8) suitable for real-time communication like video calls, live streaming and online gaming.
9) E.g., Email, SMS, UDP, RPC.	9) E.g., Video streaming, Voice over IP, TCP.